



Maschinelles Lernen

5. Generalisierung

Prof. Dr. Dennis Janka

Ziele dieses Kapitels

- Overfitting auf den **Trainingsdaten** erkennen und einordnen
- Overfitting auf den **Testdaten** als subtilere Fehlerquelle verstehen
- **Kreuzvalidierung** als zuverlässige Schätzung des Generalisierungsfehlers
- **Regularisierung** (Ridge, Lasso) als Mittel gegen Overfitting

Gliederung

- 5.1 Overfitting auf den Trainingsdaten
- 5.2 Overfitting auf den Testdaten
- 5.3 Kreuzvalidierung
- 5.4 Regularisierung
- 5.5 Weitere Gründe für schlechte Generalisierung

Erinnerung: Verschiedene Fehler

Trainingsfehler: Fehler, den das Modell auf den Trainingsdaten macht

Testfehler: Fehler, den das Modell auf den Testdaten macht

Generalisierungsfehler: Fehler, den das Modell auf unbekannten Daten macht

Ziel ist immer ein Modell, das den Generalisierungsfehler minimiert!



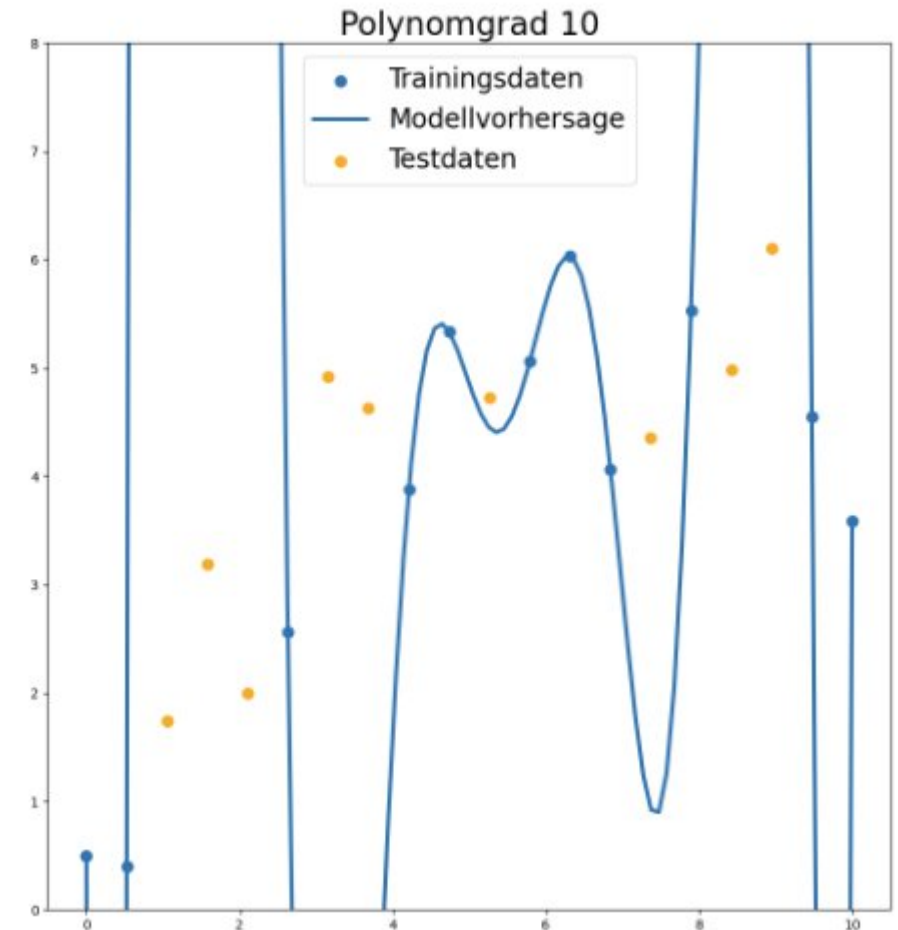
Overfitting auf den Trainingsdaten



Problem: Training minimiert den Fehler auf den *Trainingsdaten*.

Das Modell kann die Trainingsdaten sehr gut beschreiben, aber schlecht generalisieren — das nennt man **Overfitting**.

Trainings- und Testfehler sind dann weit auseinander.



Polynomielle Regression:
Overfitting bei hohem Grad

Overfitting auch bei linearen Modellen

Overfitting tritt nicht nur bei nichtlinearen Modellen auf — auch lineare Modelle mit **vielen Features** können overfitten.

Beispiel: Vorhersage der Kaltmiete in Karlsruhe mit dem Feature facilities (One-Hot-kodiert → viele Spalten)

	Trainingsdaten	Testdaten
R^2	hoch	deutlich niedriger
RMSE	niedrig	deutlich höher



Overfitting vermeiden

1. Mehr bzw. diversere Trainingsdaten beschaffen.
2. Modellkomplexität reduzieren, indem weniger Features und nichtlineare Terme benutzt werden.
3. **Regularisierung** des Modells —> später



Overfitting auf den Testdaten



Eine subtilere Fehlerquelle

In der Praxis beobachtet man oft: Das Modell performt gut auf den Testdaten — aber auf wirklich **neuen, unbekannten Daten** deutlich schlechter.



Ursache: Overfitting auf die Testdaten!

Während der Modellentwicklung werden viele Entscheidungen getroffen:

- Welche Features?
- Welche Vorverarbeitung?
- Welche Hyperparameter?

Jedes Mal wird das Modell implizit nach dem **Testfehler** ausgewählt — der Testfehler wird zum Optimierungsziel.



Warum der Testfehler kein zuverlässiger Schätzer ist



Analogie: Ein Algorithmus optimiert den Trainingsfehler → das Modell overfittet auf Trainingsdaten.

Ein/e Data ScientistIn optimiert (implizit) den Testfehler → das Modell overfittet auf Testdaten!

Konsequenz:

- Je öfter man Entscheidungen basierend auf dem Testfehler trifft, desto stärker wird der Testfehler als Schätzer für den Generalisierungsfehler verzerrt
- Der Testfehler ist ein **zu optimistischer** Schätzer für den echten Generalisierungsfehler

Lösung: Kreuzvalidierung (und/oder Verwendung des Testdatensatzes erst ganz am Ende)





Kreuzvalidierung



Idee: Mehrfaches Aufteilen

Problem: Ein einzelner Train-Test-Split liefert eine unzuverlässige Schätzung des Generalisierungsfehlers.

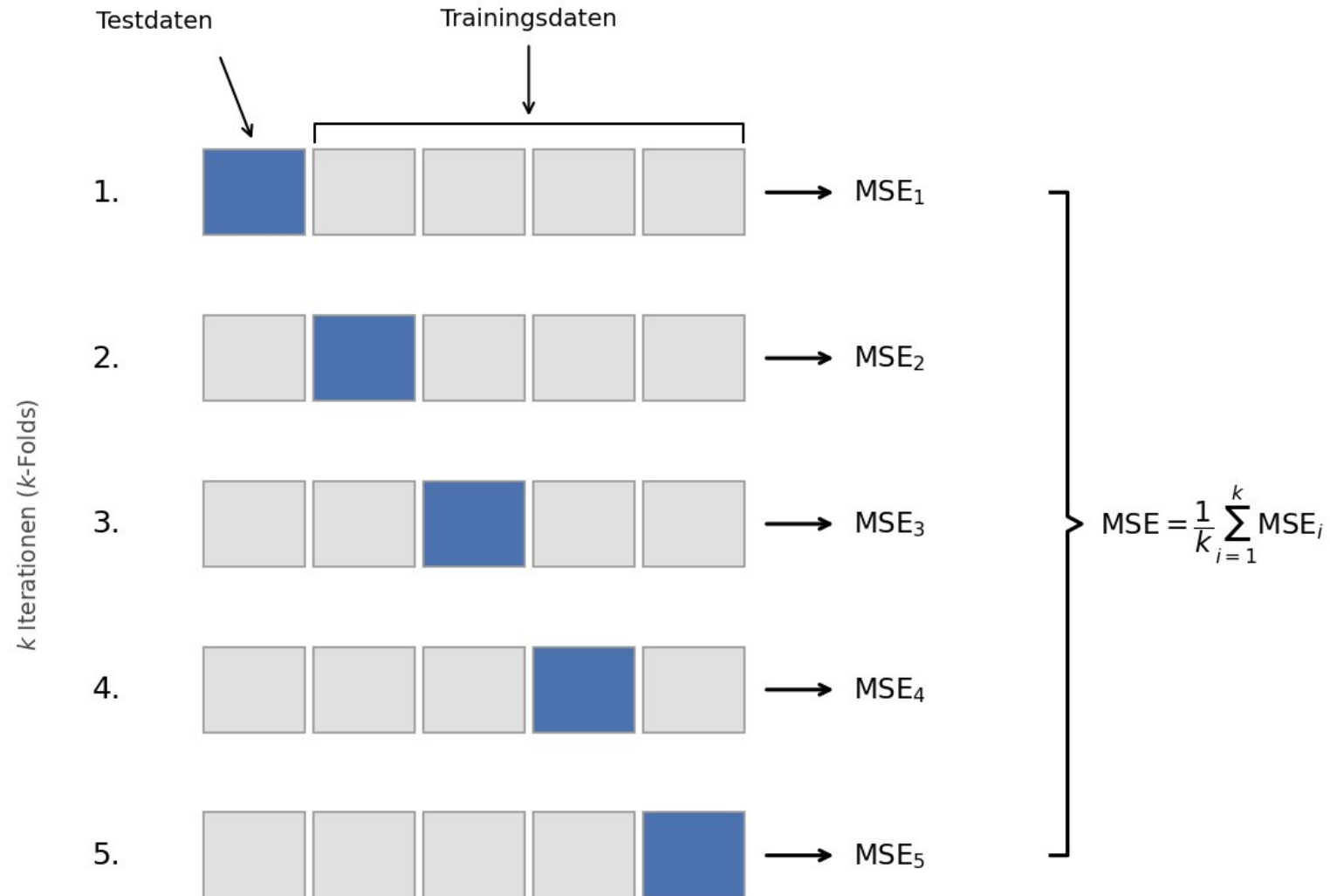
Lösung: Den Split k -mal wiederholen und die Ergebnisse mitteln — das ist **k -fache Kreuzvalidierung** (*k -fold cross-validation*).

Vorteil: Jeder Datenpunkt wird genau einmal als Testdaten verwendet → stabilere Schätzung.

Nachteil: k -mal so teuer wie ein einzelnes Training.

Typische Werte: $k = 5$ oder $k = 10$.

Ablauf der k -fachen Kreuzvalidierung



1. Datensatz wird in k gleich große **Folds** aufgeteilt
2. Fold 1 als Testdaten, restliche $k-1$ Folds als Trainingsdaten $\rightarrow$ trainieren, Fehler MSE_1 berechnen
3. Fold 2 als Testdaten, Vorgang wiederholen $\rightarrow MSE_2$
4. ... insgesamt k Mal
5. Schätzung des Generalisierungsfehlers:
6. $MSE_{CV} = \frac{1}{k} \sum_{i=1}^k MSE_i$

Kreuzvalidierung in sklearn

```
from sklearn.model_selection import cross_validate
```

```
scores = cross_validate(  
    model,  
    X, y,  
    cv=5,  
    scoring="r2"  
)
```

```
print(scores["test_score"]) #  $R^2$  für jeden Fold  
print(scores["test_score"].mean()) # Mittlerer  $R^2$ 
```

- cv=5: 5-fache Kreuzvalidierung
- scoring: beliebige sklearn-Metrik
- Kein separater train_test_split nötig



Regularisierung



Was ist Regularisierung?

- **Idee:** Modelle absichtlich weniger flexibel machen, um Overfitting zu verhindern.
- Bei linearen Modellen: Die **Koeffizienten schrumpfen** — sie werden gezwungen, näher an Null zu bleiben.
- Warum hilft das? Große Koeffizienten bedeuten, dass kleine Änderungen in den Features zu großen Änderungen in der Vorhersage führen → hohe Varianz → Overfitting.
- Durch Schrumpfen der Koeffizienten wird die **Varianz reduziert** auf Kosten eines etwas höheren **Bias** — die Gesamtperformance auf Testdaten verbessert sich.
- Die zwei wichtigsten Methoden: **Ridge** (ℓ_2) und **Lasso** (ℓ_1).

Ridge Regression (ℓ_2 -Regularisierung)

Statt der normalen Loss-Funktion wird minimiert:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \sum_{j=1}^p w_j^2$$

Der zweite Term $\lambda \|\mathbf{w}\|_2^2$ ist der **Regularisierungsterm** (Strafterm).

- $\lambda = 0$: normales lineares Regression (kein Effekt)
- $\lambda \rightarrow \infty$: alle Koeffizienten $\rightarrow 0$

Der Hyperparameter $\lambda > 0$ steuert die **Stärke der Regularisierung**.

Zwei Ziele werden gegeneinander abgewogen: 1. Daten gut fitten (MSE-Term) 2. Koeffizienten klein halten (Strafterm)

Ridge: Einfluss auf Performance

Immoscout-Beispiel: Karlsruhe, 61 Features, 383 Datenpunkte

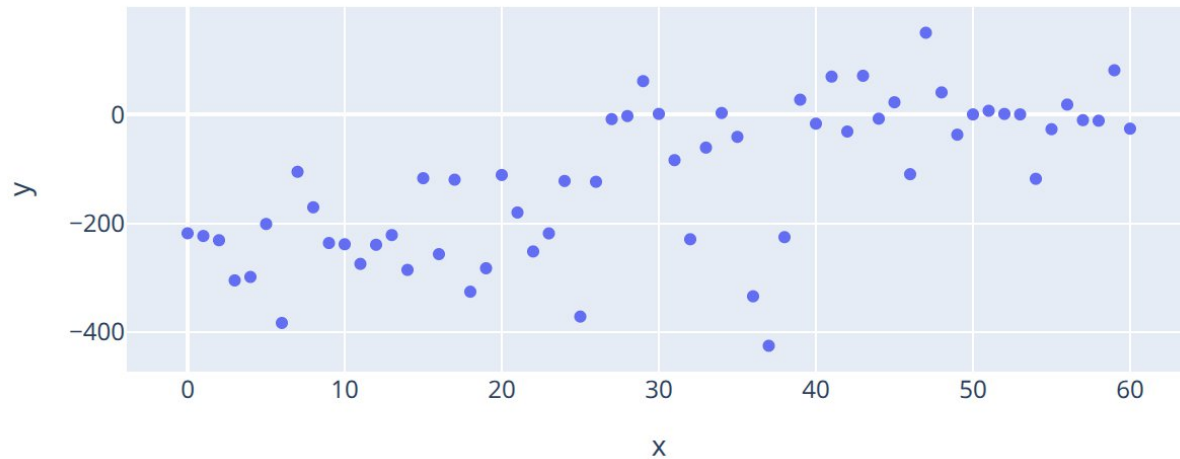
	Lin. Reg.	Ridge ($\lambda = 10$)
R^2 Train	0.789	0.768
R^2 Test	0.719	0.759

Beobachtung:

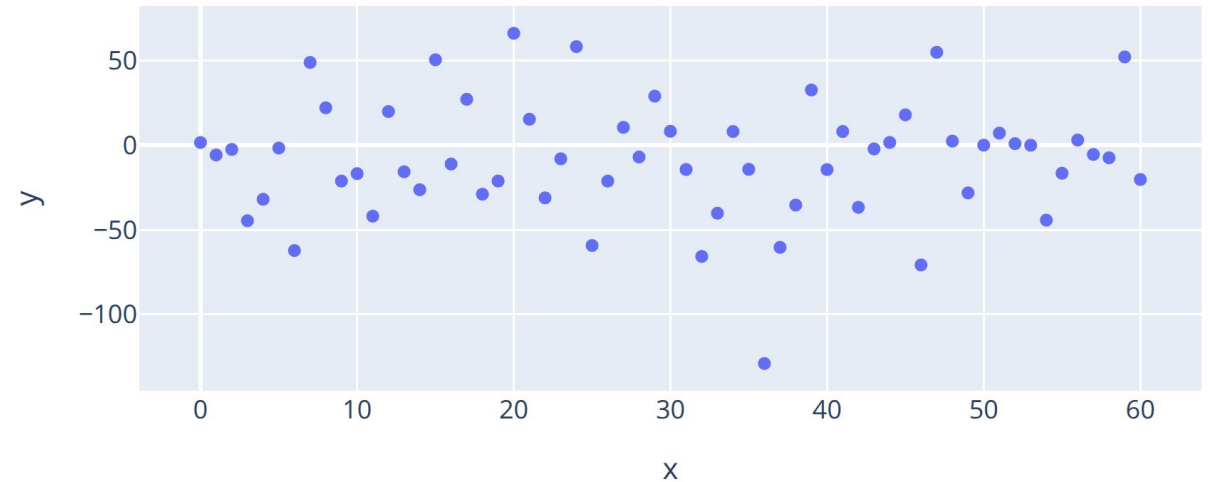
- Trainingsfehler **steigt** (Modell wird weniger flexibel)
- Testfehler **sinkt** (bessere Generalisierung)

Ridge: Einfluss auf Koeffizienten

Koeffizienten Lineare Regression



Koeffizienten Ridge Regression



- Koeffizienten werden „geschrumpft“, d.h. liegen näher an Null
- Verantwortlich: Regularisierungsterm

Lasso (ℓ_1 -Regularisierung)

Beim **Lasso** (Least Absolute Shrinkage and Selection Operator) wird die ℓ_1 -Norm als Strafterm verwendet:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \sum_{j=1}^p |w_j|$$

- Ähnlicher Effekt wie Ridge: Koeffizienten werden kleiner, Overfitting wird reduziert
- **Wichtiger Unterschied:** Lasso setzt viele Koeffizienten auf **genau Null**

Lasso führt automatisch eine **Featureauswahl** durch!

Lasso: Einfluss auf Performance

Immoscout-Beispiel: Karlsruhe, 61 Features, 383 Datenpunkte

	Lin. Reg.	Lasso ($\lambda = 2$)	Ridge ($\lambda = 10$)
R^2 Train	0.789	0.757	0.768
R^2 Test	0.719	0.755	0.759

Beobachtung:

- Einfluss auf Performance ähnlich wie bei Ridge

Lasso: Featureauswahl durch Sparsity

Ridge: Alle Koeffizienten werden geschrumpft, aber selten auf genau Null

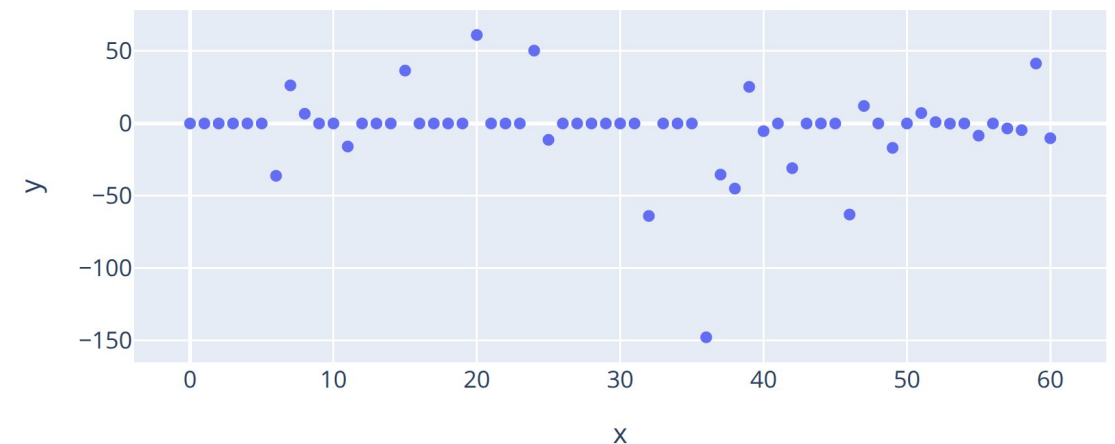
Lasso: Viele Koeffizienten werden auf **exakt Null** gesetzt

→ Lasso wählt automatisch eine Teilmenge der relevanten Features aus

Das ist besonders nützlich wenn:

- Viele Features vorhanden, aber nur wenige relevant sind
- Man wissen möchte, welche Features wirklich wichtig sind

Koeffizienten Lasso



Koeffizienten Lasso: viele genau Null

	Ridge (ℓ_2)	Lasso (ℓ_1)
Strafterm	$\lambda \sum w_j^2$	$\lambda \sum w_j $
Koeffizienten	geschrumpft, $\neq 0$	viele = 0
Featureauswahl	nein	ja (automatisch)
Differenzierbarkeit	überall	nicht in $w_j = 0$
Algorithmen	effizienter	etwas aufwändiger



In der Praxis: Beide werden verwendet — je nach Anwendungsfall liefert mal die eine, mal die andere Methode bessere Ergebnisse.



Wahl des Hyperparameters λ

λ steuert den **Bias-Varianz-Tradeoff**:

- λ zu klein $\rightarrow$ wenig Regularisierung $\rightarrow$ Overfitting möglich
- λ zu groß $\rightarrow$ zu starke Regularisierung $\rightarrow$ Underfitting

Wie wählt man λ ?

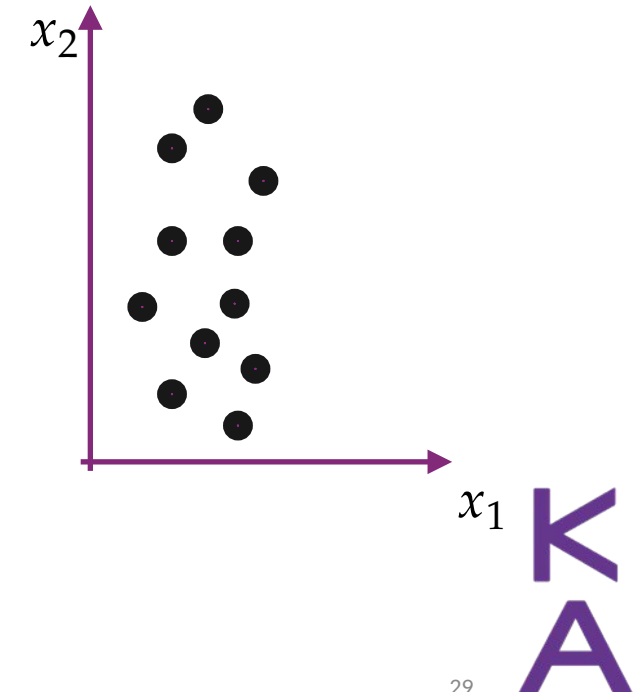
Kreuzvalidierung über mehrere λ -Werte: Wähle das λ , das den kleinsten CV-Fehler liefert.

In sklearn gibt es dafür fertige Klassen:

- RidgeCV — Ridge mit eingebauter Kreuzvalidierung
- LassoCV — Lasso mit eingebauter Kreuzvalidierung

Vor der Regularisierung: Features Skalieren

- Features können generell sehr unterschiedliche Wertebereiche aufweisen.
- Beispiel: Vorhersage des Mietpreises einer Wohnung
 - x_1 Wohnung hat Balkon $\in [0, 1]$
 - x_2 Größe der Wohnung $\in [30, 300] \text{ m}^2$
 - x_3 Baujahr $\in [1600, 2021]$
- Wertebereich von Feature x_2 ist um Faktor 30 – 300 größer als von Feature x_1
- Wertebereich von Feature x_3 ist um Faktor 5 – 60 größer als von Feature x_2



Wo kann das problematisch sein?

Was bedeutet das für die Koeffizienten der linearen Regression?

1. Regularisierung:

- Der Betrag der Koeffizienten w spiegelt bei unskalierten Features nicht ihre Wichtigkeit wieder, da alle Koeffizienten unterschiedliche „Einheiten“ haben
- Alle Koeffizienten werden aber mit dem gleichen Parameter λ skaliert
- —> Betragsgroße Koeffizienten werden bevorzugt reduziert

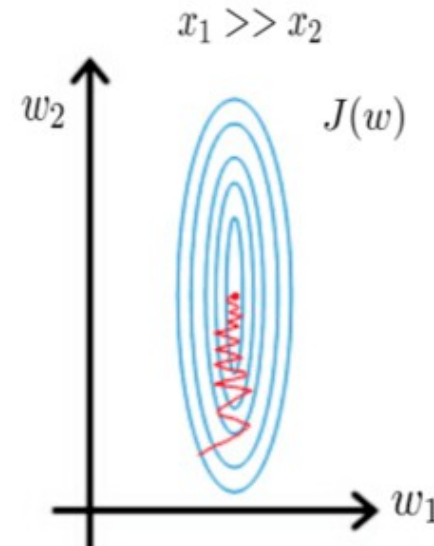
Wo kann das problematisch sein?

2. Gradientenabstiegsverfahren:

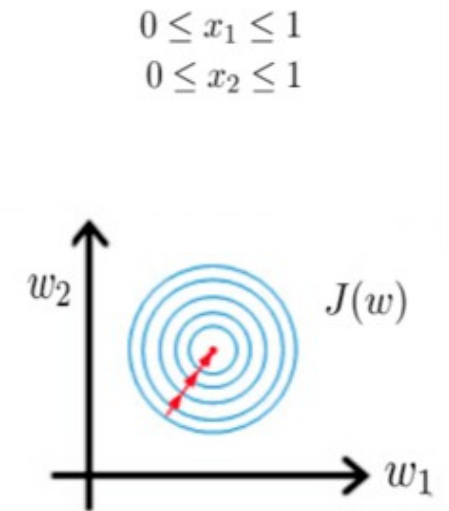
- Feature mit größerem Wertebereich dominiert den Gradienten.
- Gradient Descent springt vor und zurück.
- Verfahren konvergiert langsamer.

—> Features müssen in den gleichen Wertebereich skaliert werden.

Gradient descent
without scaling



Gradient descent
after scaling variables



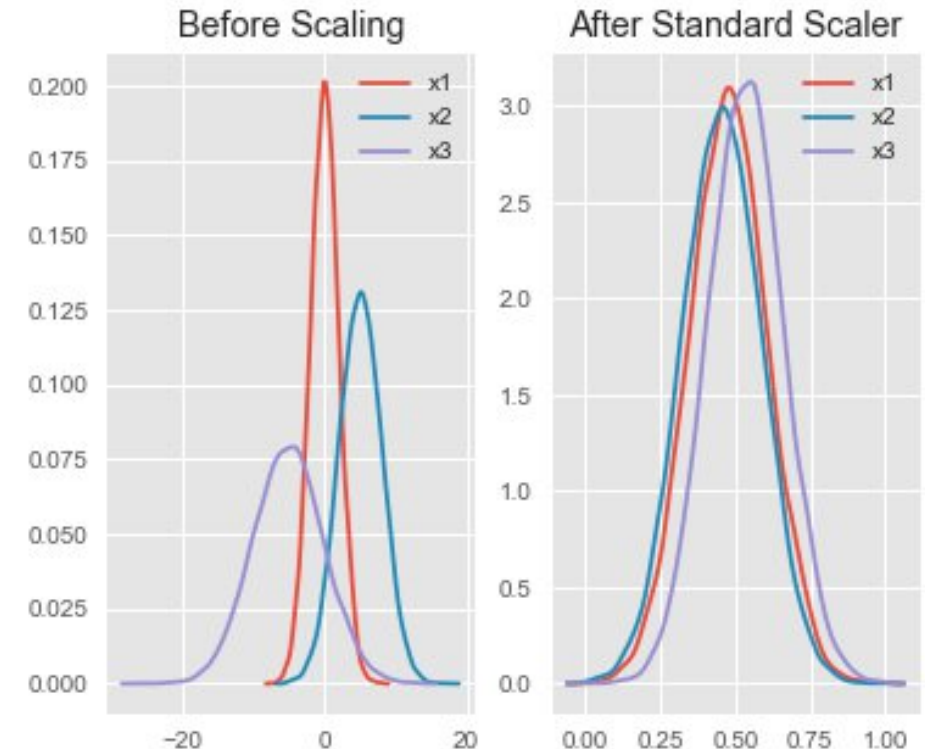
Skalierung von Features: Standardisierung

Ziel: Jedes Feature hat

1. den (arithmetischen) Mittelwert 0.
2. eine Standardabweichung von $\sigma = 1$.

→ Transformiere Feature x zu x' wie folgt:

$$x' = \frac{x - \bar{x}}{\sigma} = \frac{x - \frac{1}{N} \sum_{i=0}^N x_i}{\sqrt{\frac{1}{N} \sum_{i=0}^N (x_i - \bar{x})^2}}$$



Min-Max Normalisierung

Transformiere Feature x zu x' wie folgt:

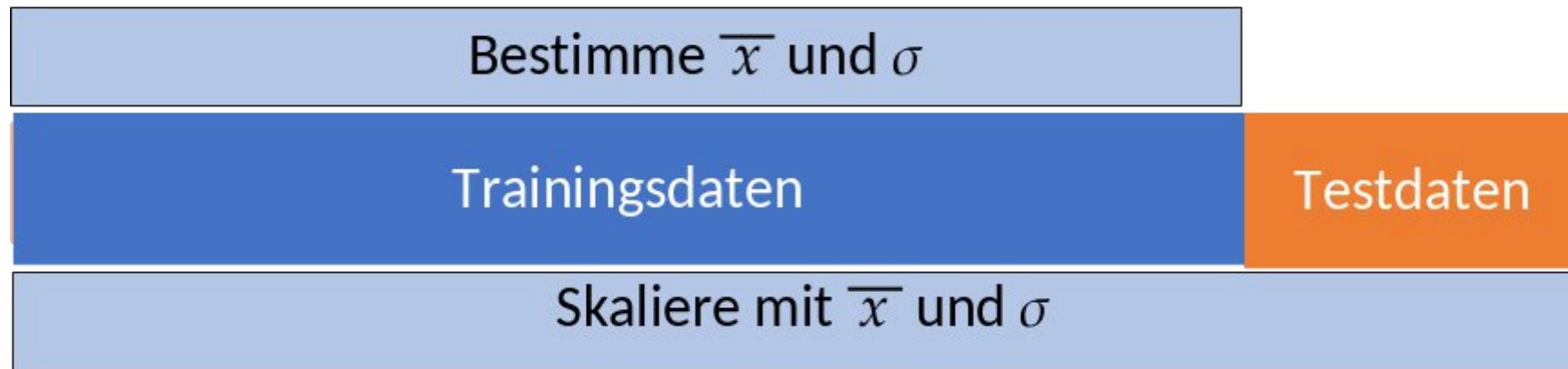
$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Effekt:

- Das Minimum des Features wird zu einer 0.
- Das Maximum des Features wird zu einer 1.
- Min-Max-Normalisierung ist nicht sehr robust gegen Outlier.

Skalierung von Features

Die Parameter $\bar{x}$ und σ (bzw. $\min(x)$ und $\max(x)$) werden auf den **Trainingsdaten** bestimmt und werden auf **Trainingsdaten und Testdaten** zur Skalierung verwendet:



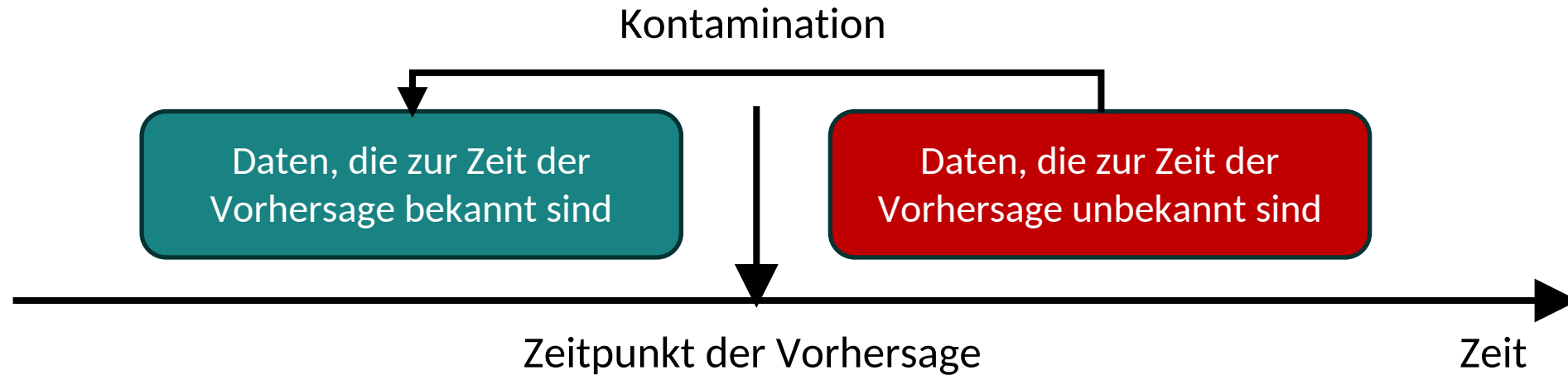
Erinnerung: Testdaten dürfen während des Modelltrainings (das umfasst auch Bestimmen der Werte $\bar{x}$ und σ) nicht verwendet werden.



Wodurch kann Generalisierung noch verhindert werden?



Data leakage



Training auf kontaminierten Daten führt dazu, dass die Performance der Modelle überschätzt wird

Data leakage kann in verschiedenen Phasen des Workflows passieren

Beispiel: Data Leakage während des Splits

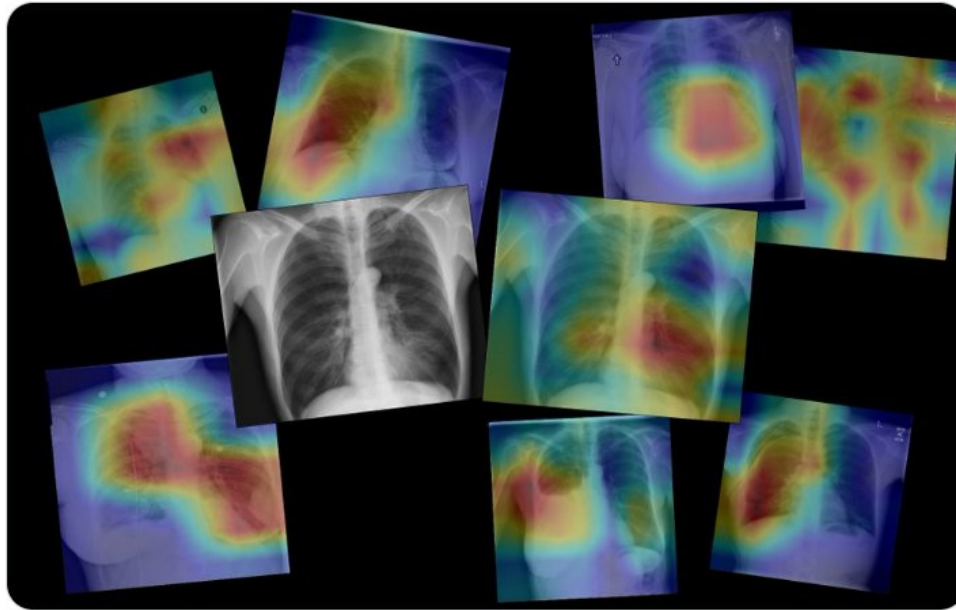


Andrew Ng ✓
@AndrewYNg

...

Our full paper on Deep Learning for pneumonia detection on Chest X-Rays. @pranavrajpurkar @jeremy_irvin16 @mattlungrenMD
arxiv.org/abs/1711.05225

Post übersetzen



3.1. Training

We use the ChestX-ray14 dataset released by Wang et al. (2017) which contains 112,120 frontal-view X-ray images of 30,805 unique patients. Wang et al. (2017) annotate each image with up to 14 different thoracic pathology labels using automatic extraction methods on radiology reports. We label images that have pneumonia as one of the annotated pathologies as positive examples and label all other images as negative examples for the pneumonia detection task. We randomly split the entire dataset into 80% training, and 20% validation.

Before inputting the images into the network, we downscale the images to 224×224 and normalize based on the mean and standard deviation of images in the ImageNet training set. We also augment the training data with random horizontal flipping.

6:09 vorm. • 16. Nov. 2017 aus Mountain View, CA



Data Leakage I: Schlechter Split

- Dieselben Patienten in Training und Testset!
- Problem: Modell könnte Features von einzelnen Patienten lernen und hätte somit einen „Wissensvorsprung“, wenn dieselben Patienten im Testset wieder auftauchen
- Besser: Nach Patienten, nicht nach Bildern splitten

Data Leakage II: Stellvertreter-Label



Vorhersage: AnnualIncomeUSD (Label)

EmployeeID	Title	ExperienceYears	MonthlySalaryGBP	AnnualIncomeUSD
315981	Data Scientist	3	5,000.00	78,895.44
4691	Data Scientist	4	5,500.00	86,784.98
23598	Data Scientist	5	6,200.00	97,830.35

Attribut MonthlySalaryGBP ist (lineare) Funktion des Labels, damit perfekte Vorhersage möglich

—> Label, Duplikat des Labels oder „Stellvertreter-Labels“ müssen entfernt werden

Data Leakage III: Was ist hier das Problem?



Vorhersage: Bei der Vergabe eines Kredites soll vorhergesagt werden, ob die Person kreditwürdig ist.

Education	Married	AnnualIncome	Purpose	LatePaymentReminders	IsBadLoan
1	Y	80k	Car Purchase	0	0
3	N	120k	Small Business	3	1
1	Y	85k	House Purchase	5	1
2	N	72k	Marriage	1	0

Data Leakage IV: Bei der Vorverarbeitung



Data Leakage kann außerdem bei der Datenvorverarbeitung auftreten:

- Skalierung von Features
- Auffüllen von NaN-Werten
- One-Hot-Encoding

Zusammenfassung

- **Generalisierungsfehler:** Fehler auf unbekannten Daten — das eigentliche Ziel
- **Overfitting auf Trainingsdaten:** Modell zu flexibel → Trainingsfehler $\ll$ Testfehler
- **Overfitting auf Testdaten:** Viele Entscheidungen nach Testfehler → Testfehler zu optimistisch
- **Kreuzvalidierung:** k -maliges Aufteilen → robuste Schätzung des Generalisierungsfehlers
- **Ridge (ℓ_2):** Strafterm $\lambda \|\mathbf{w}\|_2^2$ → Koeffizienten geschrumpft, kein Feature wird entfernt
- **Lasso (ℓ_1):** Strafterm $\lambda \|\mathbf{w}\|_1$ → viele Koeffizienten = 0, automatische Featureauswahl
- **Data Leakage:** Modell benutzt Informationen, die zum Zeitpunkt der Vorhersage unbekannt sind